

Reward Design with Generalized Prioritized Sweeping in Taxi-v4

Student: <your name>

Course: Reinforcement Learning

Date: June 2026

1. AI Use Disclaimer

During this project I used AI assistance, mainly ChatGPT/Codex, for debugging, code review, report structuring, and clarification of reinforcement learning concepts. The AI helped identify implementation risks, explain the project requirements, improve the experiment runner, and draft parts of this report. The final algorithmic choices, experiment design, interpretation, and submission responsibility remain mine. AI assistance was also used to generate report wording and to organize the project into a reproducible form.

2. Work Beyond the Minimum Requirements

The project satisfies the minimum requirements and includes several additions:

- Five reward designs are implemented instead of four.
- The fifth reward is a custom Composite Remaining Path reward.
- The code saves trained policies as model artifacts under `results/models`.
- The project includes saved-policy visualization through `python main.py --visualize-saved`.
- The project includes a step-by-step explanatory simulator in `explain_simulation.py`.
- The experiment runner generates a limited, submission-friendly set of 8 plots.

Important note: the current code contains the updated Composite Remaining Path reward as `reward_5`. If results were generated before this update, the reward-comparison plots should be regenerated before final submission using `python main.py`.

3. Environment

The environment is Gymnasium Taxi-v4 using the default Taxi map. The code attempts to set `is_rainy=False`, as required. If the installed Gymnasium version does not support this argument, the code falls back to the standard Taxi-v4 environment while preserving the same deterministic Taxi task.

Taxi-v4 has:

- 500 discrete states.
- 6 actions: south, north, east, west, pickup, dropoff.

- Passenger locations: Red, Green, Yellow, Blue, or inside the taxi.
- Four possible destinations.
- The standard base reward from the environment.

The state is decoded as `(taxi_row, taxi_col, passenger_location, destination)`. This representation is used both by the reward functions and by the kernel-based generalization mechanism.

4. Algorithm

The base algorithm is tabular SARSA with epsilon-greedy action selection. The update rule is:

```
Q(s_t, a_t) <- Q(s_t, a_t) + alpha * delta_t
delta_t = r_{t+1} + gamma * Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)
```

The agent is implemented in one class, `GeneralizedSweepingSARSAAgent`, with flags for enabling or disabling prioritized sweeping and kernel generalization. This supports the required component ablations without duplicating agent classes.

4.1 Action Selection and Exploration

The agent uses epsilon-greedy action selection. When several actions share the best Q-value, the implementation breaks ties randomly. This is important early in learning, because all Q-values begin at zero. Without random tie-breaking, the agent could repeatedly prefer the first action and explore less effectively.

Epsilon starts high and decays over time. The decay is computed dynamically:

```
epsilon_decay = 0.05 ** (1.0 / (0.6 * num_episodes))
```

This means epsilon reaches approximately 0.05 after 60% of the training horizon, regardless of whether the run uses 300, 400, or 600 episodes.

4.2 Prioritized Sweeping

The agent stores observed transitions in an internal model:

```
M(s, a) = (r, s', done)
```

After a real SARSA update, the magnitude of the TD error is used as a priority. High-priority state-action pairs are inserted into a priority queue. During planning, the agent repeatedly pops high-priority transitions and applies SARSA-style updates from the learned model.

Unlike simple Dyna-Q, this implementation does not sample model transitions uniformly. It uses prioritized sweeping, so larger TD errors receive planning attention earlier. The planning update also uses an epsilon-greedy next action, preserving the SARSA-style update required by the project.

4.3 Kernel Generalization

The generalization mechanism updates similar state-action pairs after an observed transition. The kernel is:

$$K(s, s_{\text{hat}}) = 1 / (1 + \exp(\beta * (d(s, s_{\text{hat}}) - c)))$$

where d is the Manhattan distance between taxi positions. The implementation uses:

```
beta = 4.0
c = 0.5
max_kernel_neighbors = 4
kernel_scale = 0.5
```

Two states are considered kernel-neighbors only if they share the same passenger status/location and the same destination. This follows the project warning that nearby taxi positions may represent different subtasks if the passenger or destination differs.

For runtime control, only the closest kernel neighbors are used. This keeps training practical while still allowing value updates to spread locally across the grid.

The implementation computes each neighbor's own TD error from its own stored model transition:

$$\delta_{\text{hat}} = r_{\text{hat}} + \gamma * Q(s_{\text{hat}'}, a_{\text{hat}'}) - Q(s_{\text{hat}}, a)$$

This matches the mathematical requirement that generalized updates should use the neighbor trajectory rather than reusing the original TD error.

4.4 Design Choice: Kernel on Movement Actions

Kernel generalization is applied only to movement actions: south, north, east, and west. It is not applied to pickup or dropoff actions.

The reason is semantic consistency. Moving east in two nearby taxi states has a similar meaning if passenger status and destination match. Pickup and dropoff are different: their legality and effect depend sharply on being exactly at the passenger or destination location. Generalizing pickup/dropoff across nearby states could spread invalid action values into states where the action is illegal. Therefore, the kernel is restricted to movement actions.

4.5 Design Choice: Kernel Outside Planning

In the current configuration, the full framework uses SARSA, prioritized sweeping, and kernel updates after real observed transitions. Kernel updates are not applied inside each planning step by default. This was chosen to reduce runtime and avoid repeated local propagation from simulated transitions. The resulting full setting still compares all three components: online SARSA learning, model-based prioritized planning, and local value generalization.

5. Reward Designs

Five rewards are implemented. Rewards 3, 4, and 5 are custom designs.

5.1 Base Environment Reward

This reward uses the Taxi-v4 reward directly:

```
R = R_base
```

It is the natural baseline and preserves the original MDP.

5.2 Sparse Reward

The sparse reward ignores most feedback and rewards only successful completion:

```
R = 1.0 if successful dropoff  
R = 0.0 otherwise
```

This reward tests how difficult learning becomes when the agent receives almost no intermediate guidance.

5.3 Reward 3: Dense Target Progress Reward

Reward 3 gives dense feedback for moving toward the current subgoal. Before pickup, the current target is the passenger. After pickup, the current target is the destination.

```
progress = old_distance_to_current_target - new_distance_to_current_target  
  
R = 0.75 * progress - 0.05  
+ 2.0 for successful pickup  
+ 5.0 for successful dropoff  
- 2.0 for illegal pickup/dropoff  
- 0.5 for wall bump
```

The intuition is that the agent should receive immediate feedback when it moves in a useful direction. This makes learning easier than sparse reward because useful navigation behavior is reinforced before the final dropoff.

5.4 Reward 4: Subgoal Safety Reward

Reward 4 emphasizes safe task completion. It uses a small time penalty, strong subgoal rewards, and explicit penalties for illegal or unproductive behavior.

```
R = -0.10  
+ 3.0 for successful pickup  
+ 10.0 for successful dropoff  
- 4.0 for illegal pickup/dropoff  
- 1.0 for wall bump  
- 0.25 when moving away from the current target
```

This reward is designed to strongly discourage illegal actions while still maintaining pressure to finish the task quickly. It is especially useful in Taxi because pickup and dropoff actions can easily be attempted in invalid states.

5.5 Reward 5: Composite Remaining Path Reward

Reward 5 is the most global custom reward. Instead of measuring only progress toward the immediate target, it measures progress in the estimated total remaining task path.

If the passenger has not yet been picked up:

```
remaining = distance(taxi, passenger) + distance(passenger, destination)
```

If the passenger is already inside the taxi:

```
remaining = distance(taxi, destination)
```

The reward is:

```
R = (remaining_before - remaining_after) / 16.0  
- 0.02  
+ 1.0 for successful dropoff  
- 0.3 for illegal pickup/dropoff
```

This is a complete reward replacement rather than base reward plus shaping. Its purpose is to give the agent a global signal about whether the whole remaining task is getting shorter. It is more task-aware than a purely local distance reward.

One limitation is that the reward uses Manhattan distance, while Taxi has walls. Therefore, the distance is a heuristic estimate of remaining effort rather than an exact shortest-path distance. Even with that limitation, it provides a useful learning signal because it aligns broadly with the structure of the task.

6. Summary of Findings

The strongest overall lesson from this project is that reward design, planning, and exploration interact strongly.

Sparse reward is conceptually clean but difficult for the agent, because most episodes provide no useful learning signal until the final dropoff. Dense custom rewards improve learning by providing feedback during navigation, pickup, and dropoff. Reward 4 is expected to be especially stable because it explicitly penalizes illegal actions and wall bumps while rewarding subgoals. Reward 5 is more global and should be interesting to compare because it rewards shortening the whole remaining task, not only the current navigation leg.

Prioritized sweeping helps because it reuses important observed transitions. In Taxi, many states share local structure, so planning can improve sample efficiency. Kernel generalization further spreads information across nearby taxi positions that share the same passenger status and destination. The kernel is helpful when the nearby states truly represent the same subtask, but it must be limited carefully to avoid spreading values across semantically different states.

The current full run uses 5 seeds and 600 episodes. A full run observed during development took about 41.67 minutes on the local machine. This is longer than initially expected because the experiment still trains many separate agents:

Ablation: 8 configs * 5 seeds * 600 episodes
Reward comparison: 5 rewards * 5 seeds * 600 episodes
Parameter sweep: 2 rewards * 2 alpha values * 2 planning values * 5 seeds * 400 episodes

7. Experiment Setup

The default full experiment uses:

Environment: Taxi-v4, is_rainy=False when supported
Seeds: [1, 2, 3, 4, 5]
Training episodes: 600
Sweep episodes: 400
Gamma: 0.99
Default planning steps: 5
Smoothing window: 50 episodes
Confidence interval: 95% across 5 seeds

The selected metrics are:

- Environment reward.
- Success rate.
- Illegal pickup/dropoff actions.

These metrics were selected because together they answer the most important questions:

- Is the agent improving according to the real environment?
- Is the agent actually completing the task?
- Is the agent learning to avoid invalid Taxi actions?

7.1 Component Ablation

The component ablation compares four agent setups:

- Pure SARSA.
- SARSA + Prioritized Sweeping.
- SARSA + Kernel.
- Full Framework: SARSA + Prioritized Sweeping + Kernel.

This is run on:

- Base environment reward.
- Reward 4, the custom Subgoal Safety reward.

This produces the required comparison with and without planning and with and without kernel use.

7.2 Reward Comparison

The reward comparison trains the full framework with all reward designs:

- Base Environment Reward.
- Sparse Reward.
- Dense Target Progress Reward.
- Subgoal Safety Reward.
- Composite Remaining Path Reward.

The goal is to compare how reward design changes learning behavior and final policy quality.

7.3 Parameter Sweep

The parameter sweep compares two parameters:

- Learning rate α .
- Number of planning steps n .

The current sweep is intentionally compact:

```
alpha in {0.2, 0.8}
planning_steps in {2, 10}
```

It is run on:

- Base environment reward.
- Reward 4.

The compact sweep was chosen because runtime became a practical constraint. It still satisfies the requirement of sweeping two parameters, while keeping the project runnable in a reasonable amount of time.

8. Plots Included in the Submission

The code is configured to generate at most 8 plots:

1. Ablation - base - Environment Reward.
2. Ablation - base - Success Rate.
3. Ablation - reward_4 - Environment Reward.
4. Ablation - reward_4 - Success Rate.
5. Reward Comparison - Environment Reward.

6. Reward Comparison - Success Rate.
7. Parameter Sweep Heatmap - base - Environment Reward.
8. Parameter Sweep Heatmap - reward_4 - Environment Reward.

The expected plot files are:

```
results/ablation_base_env_reward.png
results/ablation_base_success.png
results/ablation_reward_4_env_reward.png
results/ablation_reward_4_success.png
results/reward_comparison_env_reward.png
results/reward_comparison_success.png
results/parameter_sweep_base_env_reward_heatmap.png
results/parameter_sweep_reward_4_env_reward_heatmap.png
```

All noisy learning curves are smoothed with a rolling mean. In the full configuration the smoothing window is 50 episodes, grouped by configuration and seed before plotting.

9. Results and Interpretation

This section should be regenerated after running the final code once with:

```
python main.py
```

The reason is that reward_5 was updated to Composite Remaining Path after earlier result files were produced. The ablation and parameter-sweep plots for base and reward_4 remain conceptually unaffected by this update, but the reward-comparison plots should be regenerated for consistency.

9.1 Ablation Results

The ablation plots compare the effect of planning and kernel generalization. The expected pattern is:

- Pure SARSA learns only from real experience and is usually slower.
- Prioritized sweeping improves sample efficiency by replaying high-error transitions.
- Kernel generalization can improve learning when nearby states share the same passenger status and destination.
- The full framework should combine both benefits, although it can also be more sensitive to hyperparameters because both planning and generalization amplify updates.

The key interpretation is not only which line is highest, but whether planning and kernel reduce the time needed to reach high success.

9.2 Reward Comparison Results

Reward comparison should be interpreted carefully because reward scales differ. Environment reward and success rate are the fairest common metrics across reward designs.

Expected qualitative behavior:

- Sparse reward may learn more slowly because it gives little signal.
- Dense progress reward should learn navigation faster.
- Subgoal safety reward should reduce illegal actions and encourage successful pickup/dropoff.
- Composite Remaining Path reward should provide a more global task signal, but may be affected by Taxi walls because it uses Manhattan distance as an approximation.

9.3 Parameter Sweep Results

The heatmaps report mean final environment reward over the last 50 episodes. They compare low versus high learning rate and low versus high planning depth.

A useful interpretation:

- Higher planning steps can improve learning by using the model more often.
- Too many planning updates may increase runtime and may amplify imperfect estimates.
- Higher alpha can learn faster but may be less stable.
- Lower alpha can be more stable but slower.

10. Best Policy Visualization

The project supports Gymnasium's Taxi visualization. After training, saved policies can be rendered with:

```
python main.py --visualize-saved
```

To visualize only one seed:

```
python main.py --visualize-saved --seed 1
```

To visualize a specific reward and agent:

```
python main.py --visualize-saved --visualize-agent full --visualize-reward reward_4 --seed 1
```

During development, the saved base full-framework policies succeeded in 4 out of 5 visualized seeds, while one seed entered a loop. This is a realistic RL outcome: different random seeds can converge to different policies, especially with exploration and function-free tabular learning. For presentation, the best successful seed should be shown, and the failed seed can be discussed as a useful example of seed variability.

11. Surprising Results and Agent Behavior

One surprising behavior observed during visualization was that a trained policy could succeed for several seeds but fail for one seed by entering a loop. This shows that average learning curves are important: a single animation may not represent the full distribution of outcomes.

Another important observation is that the Taxi visualization appears to jump between grid cells. This is expected because Taxi-v4 is a discrete grid environment. The environment does not simulate continuous vehicle movement; it displays one state after each action.

12. Exploration vs. Exploitation

Exploration is especially important early in Taxi because the agent must discover both navigation and the correct timing of pickup/dropoff. Sparse reward makes this harder: most exploratory actions produce no useful reward signal. Dense rewards reduce the exploration burden by providing intermediate feedback.

Epsilon decay balances this tradeoff. The agent begins with high exploration, then gradually becomes more exploitative after it has observed enough transitions. The dynamic decay schedule makes this behavior consistent across runs with different episode counts.

Planning changes this balance indirectly. Prioritized sweeping allows the agent to exploit its internal model more aggressively, but the planning next-action still uses epsilon-greedy SARSA selection. This keeps the planning updates aligned with the on-policy learning rule.

13. Pragmatic Engineering Choices

Several practical choices were made to keep the project runnable:

- Kernel neighbors are precomputed once at agent creation.
- Only the nearest kernel neighbors are used.
- The priority queue is bounded.
- Kernel generalization is restricted to movement actions.
- The parameter sweep is compact.
- Only 8 plots are generated.
- Models are saved so policies can be visualized without retraining.
- A --fast mode runs a single-seed short experiment for debugging.

These choices reduce runtime while preserving the core scientific comparisons required by the project.

14. Quick Start

Install dependencies:

```
pip install -r requirements.txt
```

Run the full experiment:

```
python main.py
```

Run a faster single-seed check:

```
python main.py --seed 1
```

Run a short development run:

```
python main.py --fast
```

Visualize saved trained policies:

```
python main.py --visualize-saved
```

Explain one episode step by step:

```
python explain_simulation.py --load-saved --agent full --reward base --seed 1
```

15. Project Structure

```
RL_PROJECT/  
  agent.py  
    GeneralizedSweepingSARSAAgent: SARSA, prioritized sweeping, kernel generalization.  
  
  main.py  
    Experiment runner, reward functions, plots, model saving/loading, CLI.  
  
  visualize.py  
    Gymnasium Taxi-v4 visualization with human/ansi fallback.  
  
  explain_simulation.py  
    Step-by-step explanatory simulation.  
  
  requirements.txt  
    Python package list.  
  
  results/  
    CSV files, plots, run manifest, saved models.  
  
  results/models/  
    Saved .npz Q-table artifacts.  
  
  reports/  
    Report files.
```

16. Reproducibility

The full experiment uses seeds [1, 2, 3, 4, 5]. Each episode reset uses a deterministic seed based on the experiment seed and episode number. Plots use 95% confidence intervals across seeds.

The project can also be checked quickly with one seed:

```
python main.py --seed 1
```

This is important for graders because the full experiment can take a long time.

17. Conclusion

This project demonstrates that reward design substantially changes learning behavior in Taxi-v4. Sparse reward is simple but weak as a learning signal. Dense target progress rewards help the agent learn navigation. Subgoal safety rewards add stronger structure for pickup/dropoff correctness. The Composite Remaining Path reward provides a broader task-level signal by measuring whether the entire remaining route becomes shorter.

Prioritized sweeping improves sample efficiency by focusing planning updates on high-error transitions. Kernel generalization spreads value information across nearby, semantically compatible Taxi states. Together, these components create a stronger learning system, but they also increase runtime and require careful implementation choices.

The final system is reproducible, supports 5-seed evaluation with confidence intervals, saves trained models, and provides visualization commands for presenting learned policies.